

Agent Mesh SRE — User Guide v2.1

Document version: 2.1 **Application version:** Current (surajcsibm fork, May 2026) **Last updated:** May 2026

Table of Contents

1. Introduction
 2. What Is Agent Mesh SRE?
 3. Getting Started
 4. Dashboard Layout
 5. Running a Scenario
 6. The Agent Canvas
 7. MRAL Loop Explained
 8. Sub-Agent Circles (REASON / ACT / LEARN)
 9. Approving or Rejecting Agent Actions
 10. Scenario History
 11. Topics Management Panel
 12. Live Feed & Audit Log
 13. Email Notifications
 14. Canvas Controls
 15. Authentication
 16. Deploying to Vercel
 17. Running Locally
 18. Scenarios Reference
 19. Troubleshooting
-

1. Introduction

Agent Mesh SRE is an interactive demonstration of multi-agent AI reasoning applied to Apache Kafka operations. It simulates a real SRE environment where four specialised AI agents collaborate to detect, diagnose, remediate, and report on Kafka infrastructure incidents — all without human intervention unless human-in-the-loop approval is required.

This guide covers the v2.1 fork hosted at [surajcsibm/agent-mesh-sre](https://github.com/surajcsibm/agent-mesh-sre) on GitHub. The application runs entirely client-side (no Kubernetes, no external Kafka cluster needed) using a built-in simulation engine.

2. What Is Agent Mesh SRE?

The system models four Kafka-aware agents connected through a virtual broker mesh:

Agent	Role	Colour
INTAKE	Ingests Kafka metric events (lag, offset, partition health)	Emerald green
MONITOR	Runs the MRAL reasoning loop; decides when and how to act	Violet
WRITER	Generates structured incident reports and human-readable summaries	Cyan
NOTIFY	Sends email alerts and escalation notices	Orange

The agents communicate via a simulated Kafka broker that is visible on the canvas. Metric events flow from INTAKE → BROKER → MONITOR → WRITER → NOTIFY in a directed mesh topology.

3. Getting Started

Prerequisites

- Node.js 18 or later
- npm 9 or later
- A Google account (for OAuth login) **or** an email/password account

Local setup

```
git clone https://github.com/surajcsibm/agent-mesh-sre.git
cd agent-mesh-sre
npm install
cp .env.local.example .env.local
# Edit .env.local – fill in GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET,
# NEXTAUTH_SECRET, SMTP_HOST, SMTP_USER, SMTP_PASS, SMTP_FROM, NOTIFY_TO
npm run dev
```

Open **http://localhost:3000** in your browser.

Environment variables

Variable	Purpose
GOOGLE_CLIENT_ID	Google OAuth client ID
GOOGLE_CLIENT_SECRET	Google OAuth client secret
NEXTAUTH_SECRET	Random secret for NextAuth JWT signing
NEXTAUTH_URL	Full URL of the deployment (e.g. http://localhost:3000)
SMTP_HOST	SMTP server hostname
SMTP_PORT	SMTP server port (default 587)
SMTP_USER	SMTP username / email address

SMTP_PASS	SMTP password or app password
SMTP_FROM	From address in sent emails
NOTIFY_TO	Recipient address for incident emails

4. Dashboard Layout

The dashboard uses the **Arctic Clean** theme: a light blue-grey page background (#f0f4f8), deep navy navigation (#1e3a5f), emerald accent (#1D9E75), and white surface cards.

The layout has three vertical regions:

Left sidebar — Scenario selector at the top. The top four pinned scenarios are always visible. Six additional scenarios appear in a scrollable list below them. Beneath the scenarios is the **Kafka Topics panel** showing live topic health and a paginated list of all topics. Click any scenario card to run it.

Centre canvas — The Agent Canvas showing all four agents as circular nodes connected to a central Broker node. Sub-agent ephemeral circles appear below the MONITOR node during active MRAL phases.

Right sidebar — Tabbed panel with three sections: - **Live Feed** — Real-time event stream from all agents - **Topics** — Kafka topic health, metrics, and management actions - **Audit** — Full structured audit log with timestamps

A **Live Activity Banner** slides in just below the top navigation whenever any agent is actively processing. It shows the agent name, current action, and a detail snippet.

5. Running a Scenario

1. Click any scenario card in the left panel.
2. When the scenario starts, its relevant Kafka topics are **automatically created** in the Topics panel (or updated with scenario-specific lag and health values if they already exist). This makes the Topics panel instantly reflect the incident being simulated.
3. Watch the Live Activity Banner appear and agents animate on the canvas.
4. When MONITOR reaches the **AWAITING** phase (human-in-the-loop), an approval card appears in the centre of the dashboard.
5. Read the reasoning, confidence score, and proposed action, then click **Approve** or **Reject**.
6. After the scenario completes, the Scenario End Modal appears with a full summary.
7. Close the modal — the result is automatically saved to the Scenario History bar at the bottom of the page.

You can re-run any scenario as many times as you like. Each run creates a new history entry.

6. The Agent Canvas

The canvas uses **React Flow v12** (@xyflow/react) with circular custom nodes. Each node shows:

- The agent name and a status label (IDLE / MONITOR / REASON / AWAITING / ACT / LEARN)
- A live pulse animation when active
- A **Kill** button to terminate the agent mid-run
- A **Restart** button to bring a killed agent back online

Nodes are not draggable. The canvas supports pan and zoom using standard trackpad/mouse gestures.

Canvas size controls — Two small – and + buttons appear in the top-right corner of the canvas panel. Click them to shrink or grow the canvas height in 80 px increments (minimum 380 px, maximum 960 px).

Edges between nodes animate with moving dashes while data is flowing. Edge labels show the message type being passed.

Node colours

Node	Colour
INTAKE	Emerald #1D9E75
MONITOR	Violet #7c3aed
WRITER	Cyan #0891b2
NOTIFY	Orange #f97316
BROKER	Navy #1e3a5f

7. MRAL Loop Explained

MRAL stands for **Monitor** → **Reason** → **Act** → **Learn**. It is the core reasoning cycle that the MONITOR agent executes for every incident.

Phase	What happens
MONITOR	INTAKE publishes lag/metric events; MONITOR subscribes and detects anomalies
REASON	MONITOR analyses root cause, builds a confidence score, identifies candidate actions
AWAITING	MONITOR pauses and surfaces a human-approval card if confidence < threshold
ACT	After approval (or auto-approval above threshold), MONITOR invokes the

chosen MCP tool

LEARN MONITOR records a lesson — what worked, what to do differently next time

8. Sub-Agent Circles (REASON / ACT / LEARN)

During an active MRAL cycle, a smaller ephemeral circle appears below the MONITOR node connected by a dashed animated edge labelled “spawn”:

- **REASON circle** (violet) — appears during reasoning and while awaiting approval. Shows a snippet of the root-cause analysis or “Awaiting approval...”
- **ACT circle** (orange) — appears while an action is executing. Shows the action detail.
- **LEARN circle** (green) — appears while the lesson is being recorded. Shows the lesson note snippet.

Only one sub-circle is visible at a time. It disappears automatically when the phase transitions. The dashed border and pulse animation visually distinguish sub-agents from primary agents.

9. Approving or Rejecting Agent Actions

When MONITOR reaches the AWAITING phase, an approval card appears in the centre of the dashboard. The card shows:

- The scenario name and detected root cause
- Confidence score as a percentage
- The proposed MCP tool action and its parameters
- A summary of what will happen if approved

Click **Approve** to let the agent execute the action, or **Reject** to cancel it. In both cases, the MRAL loop continues to the LEARN phase to record the outcome.

10. Scenario History

Every completed scenario run is saved to a **persistent history bar** at the bottom of the dashboard. History survives page refreshes and browser restarts because it is stored in `localStorage`.

Clicking any history entry opens a **Summary Modal** showing:

- Scenario name and result (approved / rejected)
- Consumer lag before and after
- MONITOR’s reasoning, action taken, and lesson recorded
- A **Live Feed Events** table with timestamped entries from the run

The history retains up to 30 entries; oldest entries are dropped when the limit is reached.

A **Clear History** button in the history bar wipes all saved entries.

11. Topics Management Panel

The **Topics** panel in the left sidebar (and also accessible via the right sidebar Topics tab) shows all Kafka topics being simulated. Each topic card displays:

- Topic name and partition count
- Consumer lag (with ▲/▼ trend arrow when rising/falling sharply)
- Messages per second
- Health badge: **Healthy** (green), **Degraded** (amber), or **Critical** (red)

Auto-create on scenario run

When you click a scenario, the system automatically **creates or updates** the Kafka topics relevant to that scenario in the panel. For example, clicking “Consumer Lag Spike” instantly surfaces `ops.kafka.metrics.v1` with critical lag, and `ops.requests.v1` as healthy. If a topic with the same name already exists, its lag and health metrics are updated to match the scenario’s initial conditions. This means the Topics panel always reflects the incident being simulated without any manual setup.

Topic pagination

The panel shows the **latest 20 topics** sorted by severity (critical first, then degraded, then healthy; within each group by highest lag first). A “+ N more topics ↓” button appears at the bottom when more than 20 topics exist. Each click loads 20 additional topics. The list is fully scrollable within a fixed-height container.

Topic actions

Heal Topic — Triggers a dedicated MRAL healing cycle for an unhealthy topic. MONITOR will reason about the degradation, propose a remediation, and ask for approval before acting.

Create Topic — Opens a form to add a new simulated topic with a custom name, partition count, replication factor, retention hours, and description. Duplicate names are rejected with a validation error.

Copy Topic — Duplicates an existing topic with a “-copy” suffix. The copy starts with the same configuration but zero lag.

12. Live Feed & Audit Log

The **Live Feed** tab shows a real-time stream of events as they occur during a scenario run. Each entry shows:

- Timestamp
- Event type (colour-coded: blue = publish, green = consume, violet = reasoning, orange = tool-call, amber = approval, cyan = lesson, pink = notification)
- Agent name
- A human-readable summary

Events are prepended to the top of the list so the most recent activity is always visible without scrolling.

The **Audit** tab shows the same events in a more structured table format suitable for review after a scenario completes.

13. Email Notifications

When a scenario completes and the NOTIFY agent fires, an HTML email is sent to the address configured in NOTIFY_TO. The email includes:

- Scenario name and overall result
- Consumer lag delta (before → after)
- MONITOR's reasoning summary
- Action taken and its outcome
- Lesson recorded
- A full **Live Feed Events** table with every event from the run

If the SMTP credentials are not configured, the Scenario End Modal will display an email error notice but the scenario run is otherwise unaffected.

14. Canvas Controls

Control	Location	Action
– button	Top-right of canvas	Decrease canvas height by 80 px
+ button	Top-right of canvas	Increase canvas height by 80 px
Kill button	Inside agent node	Terminate agent immediately
Restart button	Inside agent node (killed state)	Bring agent back online
Pan	Click and drag canvas	Move the viewport
Zoom	Scroll wheel / pinch	Zoom in / out
Fit View	Automatic on load	Centres all nodes

15. Authentication

The application requires sign-in before the dashboard is accessible.

Google OAuth — Click “Continue with Google” on the login page. You will be redirected to Google’s sign-in flow and returned to the dashboard automatically.

Email / password — Enter your email and password. If you don’t have an account, use the Register link. Forgot-password emails are sent via the configured SMTP connection.

Session tokens are managed by **NextAuth.js**. Sessions expire after 30 days of inactivity.

16. Deploying to Vercel

The easiest way to run the application publicly is to deploy it to Vercel.

1. Push your fork to GitHub.
2. Import the repository at vercel.com/new.
3. Add all environment variables from `.env.local` in the Vercel project settings.
4. Set `NEXTAUTH_URL` to your Vercel deployment URL (e.g. `https://agent-mesh-sre.vercel.app`).
5. Click Deploy. Vercel builds and deploys automatically.

Subsequent pushes to the main branch trigger automatic re-deploys.

17. Running Locally

```
npm run dev      # Development server on http://localhost:3000
npm run build    # Production build (checks for type/lint errors)
npm run start    # Serve the production build locally
```

TypeScript type errors in third-party library types are suppressed via `ignoreBuildErrors: true` in `next.config.mjs` so that production builds always succeed.

18. Scenarios Reference

Pinned scenarios (always visible)

Scenario	KIP / Tag	Topics auto-created
Consumer Lag Spike	KIP-848	<code>ops.kafka.metrics.v1</code> (critical), <code>ops.requests.v1</code> (healthy)
KRaft Controller Failover	KRaft	<code>ops.incidents.v1</code> (healthy), <code>ops.kafka.metrics.v1</code> (degraded)

Share Group Rebalance	KIP-932	share.group.events.v1 (critical), ops.kafka.metrics.v1 (critical)
False-Positive Suppression	KIP-848	ops.kafka.metrics.v1 (degraded)

Extra scenarios (scrollable)

Scenario	Tag	Topics auto-created
Schema Registry Mismatch	Avro	schema.registry.events.v1 (degraded)
Broker Disk Saturation	I/O	ops.broker.disk.v1 (healthy), audit.payment.events.v1 (critical)
Under-Replicated Partitions	ISR	payments.transactions.v1 (degraded), payments.settlements.v1 (critical)
Producer Timeout Storm	Batch	payments.transactions.v1 (critical)
Consumer Session Timeout	GC	invoices.created.v1 (critical)
Log Compaction Lag	Compact	ops.lessons.v1 (degraded), audit.payment.events.v1 (critical)

19. Troubleshooting

Login page keeps reloading — Check that NEXTAUTH_SECRET and NEXTAUTH_URL are set correctly in your environment. For local dev, NEXTAUTH_URL=http://localhost:3000.

Email not sending — Verify SMTP credentials. For Gmail, use an App Password (not your account password) and enable “Less secure app access” or use Google Workspace SMTP relay.

Canvas is blank — The canvas uses dynamic import with `ssr: false`. If you see a blank canvas, check the browser console for React Flow errors. Ensure the browser window is at least 800 px wide.

Scenario runs but approval card never appears — Some scenarios (False-Positive Suppression) auto-approve without pausing. Only scenarios where MONITOR confidence falls below the threshold generate an approval card.

History not persisting — Ensure localStorage is available in your browser (not blocked by private/incognito mode or browser settings).

Topics don’t appear when running a scenario — Ensure you are on the latest code (git pull). The auto-create feature requires the SCENARIO_AUTO_TOPICS map and handleTrigger wrapper introduced in the v2.1 update.

Build fails on Vercel — Check that all required environment variables are set in Vercel project settings. The build uses `ignoreBuildErrors: true` but missing env vars can still cause runtime failures.